Keyboard-Centric Browser Extension (Vim-Style Navigation) Using Rust & JavaScript

Quang Vinh Dang

Department of Computer Science, Simon Fraser University

CMPT 376W - Technical Writing

Professor XXXX

Oct 16, 2025

Word counts: **1254**

Keyboard-Centric Browser Extension (Vim-Style Navigation) Using Rust & JavaScript

As computers became widely adopted, people naturally developed interaction habits around the keyboard and mouse. These two input devices became synonymous with computer usage; when most individuals think of using a computer, they instinctively imagine using both tools together. However, not all workflows rely on the mouse equally. Certain communities, particularly software developers, have built highly efficient keyboard-centric workflows that minimize or eliminate mouse use.

One of the most notable examples is Vim, a text editor designed for fluid, rapid text manipulation through keyboard commands. Vim's modal editing system and intuitive keybindings enable users to perform complex navigation and editing tasks without removing their hands from the home row. This style of interaction has grown in popularity among developers who value speed, focus, and minimal context switching.

This proposal introduces a keyboard-centric browser extension inspired by Vim's interaction model. The goal is to enable users—particularly productivity-oriented developers—to navigate and operate their browser primarily through the keyboard, reducing reliance on the mouse. By integrating Vim-style keybindings into mainstream web browsing, the extension aims to enhance workflow efficiency and provide a seamless experience between coding environments and the web.

Why keyboard driven development?

Introduction

Keyboard-centric programming refers to a workflow in which the keyboard is the primary mode of interaction between the user and the computer. This approach emphasizes efficiency, reduced hand-movement, and uninterrupted mental flow during tasks (Fillali, 2024). This interaction style is increasingly adopted by developers who seek to maximize productivity and streamline navigation without switching input devices (Fillali, 2024).

Client’s profile (Persona)

According to discussions on Hacker News, many power users express interest in a keyboard-centric browser that supports efficient shortcuts and advanced tab management (curioussavage, 2019). However, up to now, there are only a couple browser extensions that support these such as Vinium (Crosby et al., 2011) on mainstream browsers like Chrome, Brave and Firefox but lack deep configurability and performance reliability when working with a large number of tabs. Desktop applications like qutebrowser (Bruhin & contributors, 2014) provide a more complete keyboard-driven experience, yet they are not mainstream, require significant configuration, and do not work seamlessly out of the box for most users.

Our persona is Alex. He is a software engineer who regularly works with multiple terminals, text editors, and web-based development tools. Alex values efficiency and prefers workflows that avoid context switching from his keyboard: "I live in Vim, TMUX, and Neovim all day. The browser is the only place where I still have to touch the mouse — and it feels slow

and disruptive." Alex's goals include maintaining uninterrupted keyboard workflows, navigating dozens of open tabs efficiently, and ensuring consistent keybindings across tools. He wants software that works out of the box while maintaining high performance regardless of how many tabs are open. The extension must align closely with Vim's modal interaction model, offering keybindings that behave like Vim and can be fully customized. It should also support fuzzy searching through tabs, history, and bookmarks, as these are common tasks in everyday web browsing.

Project's overview

Objective

Since we aim to integrate keyboard-driven development into the browser, our software must satisfy several conditions. First, it must provide fully keyboard-based navigation with Vim-style modal interaction. This is essential because most users who prefer keyboard-centric workflows rely on familiar keybindings and should not need to relearn them. At the same time, the extension should offer flexibility, allowing users to customize their keybindings according to their workflow preferences. Second, the extension must be easy to install and operate across major browsers, meaning it should run on multiple browser engines despite differences in JavaScript runtimes and supported APIs (mozilla.org, 2025). Finally, developers who adopt keyboard-centric tools prioritize speed and efficiency, so the extension must process keyboard input with minimal latency while maintaining a lightweight performance footprint to avoid burdening the browser.

Scope

The scope of this project focuses on delivering a browser extension that enables efficient, fully keyboard-driven navigation inspired by Vim-style interaction. The minimum viable product (MVP) will include core modal keybindings, tab navigation, scrolling, link hinting and a configuration keymap system. Support for both Chromium-based browsers and Firefox will be implemented using WebExtension standard to ensure cross-browser compatibility. The extension will prioritize performance, aiming for responsive input handling suitable for multi-tab developer workflows.

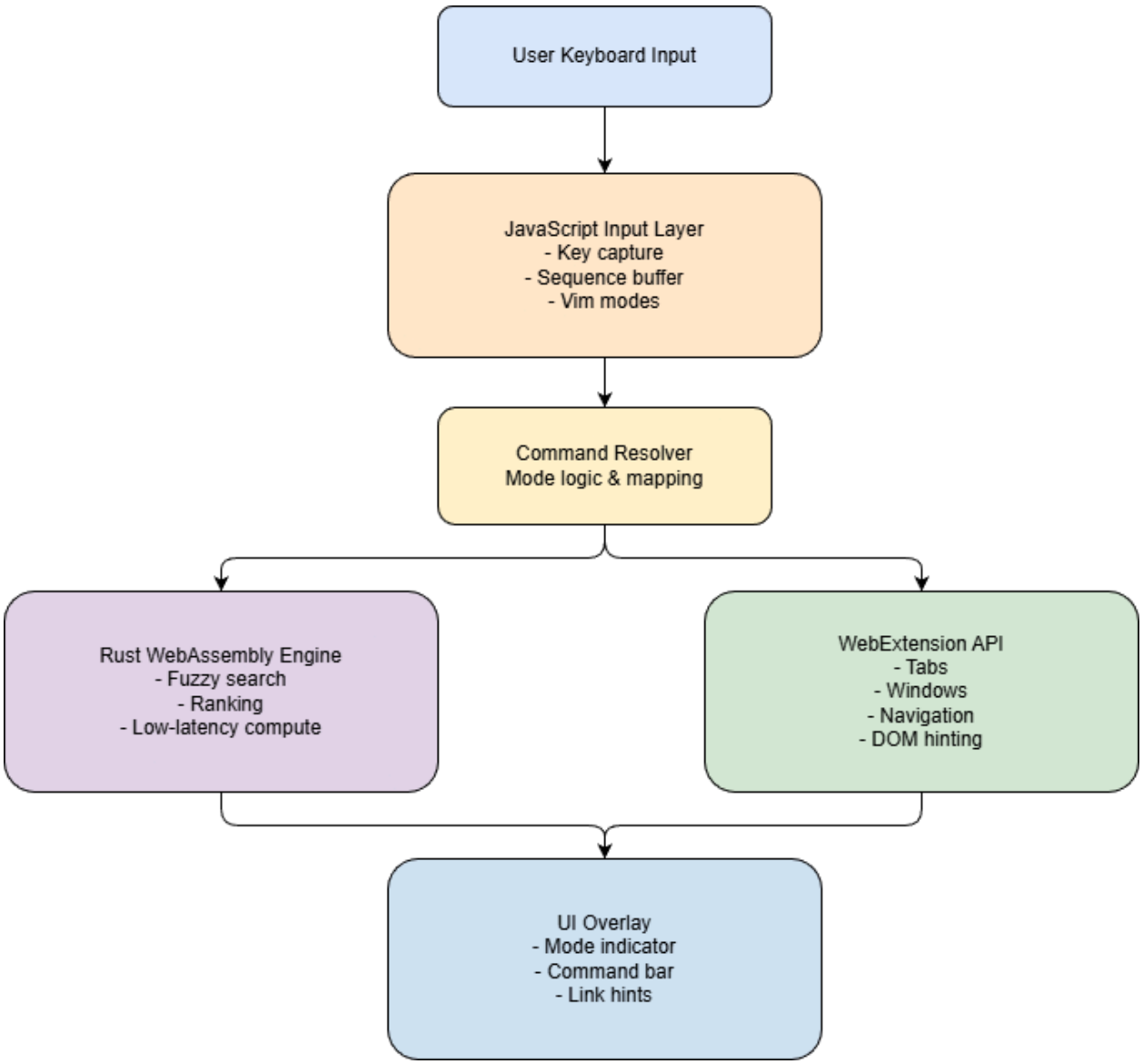
Strategy and architecture

The extension will follow a modular architecture designed for low-latency input handling and cross-browser portability. Keyboard events will be captured through an injected content script written in JavaScript, which interprets user key sequences. A WebAssembly (WASM) module compiled from Rust will be integrated to support fast tab and command searching, leveraging Rust's performance and memory-safety advantages. Because WASM modules are browser-agnostic, this approach reduces browser-specific code and improves long-term scalability. Browser interactions will be handled through the WebExtension API, abstracting differences between Chrome and Firefox (mozilla.org, 2025). A lightweight UI component will be lazily injected at startup to provide real-time mode feedback. Additionally, a configuration file

accessible through the extension's popup menu will allow users to customize keybindings and behaviors without modifying the codebase.

Figure 1

Architecture of web extension support vim's keybinding



Due to the needs of squeezing performance and speed out of JavaScript, we will have to integrate the trie data structure (Sedgewick, 2011) to our JavaScript code for listening to keybinding from the user's input. The below is a snippet of code that uses this.

```
// Build a tiny prefix trie for fast matching
function commandKeyToTokens(key) {
  // Allow array form directly, e.g., ["C-j", "j"]
  if (Array.isArray(key)) return key.slice();
  const keyStr = String(key);
  // Space-delimited tokens, e.g., "C-j j" => ["C-j","j"]
  if (keyStr.includes(" ")) return keyStr.trim().split(/\s+/);

  // Hyphen handling: treat leading modifiers (C, S) as part of first
token only.
  // Example: "C-j-j" => ["C-j", "j"] ; "C-S-j-g" => ["C-S-j", "g"]
  if (keyStr.includes("-")) {
    const segs = keyStr.split("-");
    let i = 0;
    const mods = [];
    while (i < segs.length && (segs[i] === "C" || segs[i] === "S")) {
      mods.push(segs[i]);
      i++;
    }
    if (i < segs.length) {
      const base = segs[i++];
      const first = (mods.length ? mods.join("-") + "-" : "") + base;
      const rest = segs.slice(i);
      return [first, ...rest];
    }
    return [keyStr];
  }

  // Fallback: split into individual characters (e.g., "gg" =>
["g","g"]).
  return keyStr.split("");
}

function buildCommandTrie(commands) {
  const root = { children: new Map(), handler: null };
  let maxLen = 1;
```

```
  for (const cmd of commands) {
    const tokens = commandKeyToTokens(cmd.key);
    maxLen = Math.max(maxLen, tokens.length);
    let node = root;
    for (const t of tokens) {
      if (!node.children.has(t))
        node.children.set(t, { children: new Map(), handler: null });
      node = node.children.get(t);
    }
    // Last token: store handler
    node.handler = cmd.handler;
  }
  return { root, maxLen };
}

const { root: COMMAND_TRIE, maxLen: MAX_SEQ_LEN } = buildCommandTrie(
  VIM_COMMANDS.immediate,
);
```

Benefits and feasibility

The software is designed to support users who are particular about their development workflows and prefer keyboard-driven tools integrated into their daily computing activities, including web browsing. It addresses the growing demand for efficient keyboard-centric browsing solutions and provides users with improved performance by reducing reliance on mouse interactions. Additionally, users can organize their workflow around the extension's navigation model to enhance productivity and streamline browser-based tasks.

Qualifications

I believe I am well-qualified to develop this project based on my academic background and personal hands-on development experience in both JavaScript and Rust. I have used JavaScript extensively to build personal projects as well as applications with real users, giving me practical experience in web-based software development and user-driven problem solving. I am currently taking a course in Rust and have completed a guided operating system project in Rust, which strengthened my understanding of systems programming, memory safety, and low-level execution—skills directly relevant to integrating Rust and WebAssembly into a browser extension. In addition, I am an active user of Vim and Neovim in my daily development workflow, and I have customized my own editor configuration to support highly efficient, keyboard-centric programming. This personal experience gives me direct insight into the expectations, needs, and habits of developers who prefer modal, keyboard-driven interaction. As

a user of Vimium, I also understand both the strengths and limitations of existing solutions, positioning me to build an improved, performance-focused experience for power users who desire keyboard-first browsing.

Schedule

This project will be completed over six weeks. The first week will focus on implementing the core JavaScript keyboard event listener to reliably capture and process key input. In week two, the Rust-based WebAssembly module for fast fuzzy tab search will be developed, followed by week three's implementation of a configurable keybinding system. Week four will add tab navigation features and hinting, and week five will be dedicated to testing and performance optimization to ensure smooth browser operation.

Table 1
Schedule for the development

Week	Focus area	Description/ Deliverables
1	Keyboard Input System	Implement JavaScript key event listener and key capture pipeline
2	WASM Fuzzy Search Engine	Build Rust → WASM module for fast tab and command search
3	Configurable Keybindings	Implement user-configurable keybinding system
4	Browser Navigation Features	Add tab control, navigation, and link hinting
5	Testing & Performance Tuning	Conduct performance tests, regression testing, and optimizations
6	Documentation & Packaging-	Final documentation, demo prep, and extension packaging

Conclusion

Keyboard-centric browser extensions can enhance developer productivity by enabling efficient, Vim-style navigation. They also address a rising demand for high-performance browsing tools among users who prefer keyboard-driven workflows. By integrating JavaScript and Rust, this project aims to deliver a responsive and customizable browsing experience. With a structured development plan, the extension is technically feasible and well-positioned to build a user base and address the needs of individuals seeking a streamlined, keyboard-centric web browsing experience.

Appendix 1 - AI Disclosure

I used AI tools to support this assignment. Specifically, I used ChatGPT to help identify appropriate technical resources related to cross-browser WebExtension development and to improve my understanding of WebAssembly and its role in building performant browser components. I also used AI to review and refine code snippets by checking for syntax issues, and to proofread my writing for grammar clarity and accuracy.

References

Crosby, P., Sukhar, I., Blott, S., & contributors. (2011). *Vimium* (Version 1.67) [Javascript; Browser]. https://github.com/philc/vimium

curioussavage. (2019, March 4). *What I really want is a keyboard centric power user browser* [Online post]. Hacker News. https://news.ycombinator.com/item?id=19306243

Fillali, I. (2024, September 4). *A Guide to Mogg Your Mouse-Dependent Colleagues*. DEV Community. https://dev.to/nayetwolf/keyboard-centric-computing-m68

mozilla.org. (2025, July 17). *Build a cross-browser extension*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Build_a_cross_browser_extension

Sedgewick, R. (2011). *Algorithms / Robert Sedgewick and Kevin Wayne.* (4th ed.). Upper Saddle River, NJ : Addison-Wesley.